Add/Remove a user from a database

```
- name: --- Add mysql user ---
  mysql_user:
    name: anisa
    password: 123456
    priv: 'mydb.*:ALL'
    state: present
  tags: [add_mysql_user]
```




Exercise 26: backup and restore a database

1. Bask-up the database to /tmp/db1.sql
2. Remove the database manually on both remote hosts
    1. MariaDB [(none)]> drop database db1;
3. Try to restore the database you've just removed
4. Create user anisa and assign all privileges on db1
5. Login to the mysql by anisa user and run show databases;



Find module

✓ Return a list of files based on specific criteria.

```
- name: - - - find .txt and .conf file in /tmp which
are older than 2 weeks and greater than 1K - - -
 find:
  paths: /tmp
  age: 2w
  size: 1k
  recurse: yes
  file_type: file
  pattern: '*.txt,*.conf'
```

file_type
Any, file, directory, link

Hidden
Yes, no



Local_action feature

برای انجام عملیات Bash اما روی خود سرور Ansible هست

✓ run certain tasks locally on the machine that runs Ansible.

```
- name: local_action feature
 local_action:
  module: file
  path: /home/local_file
  state: touch
 tags: [loc_act_mod]

 output:
 TASK [myproject : local_action feature] ************
 changed: [172.20.*.* -> localhost]
```




Exercise 27: find on localhost

1. Find all .txt files in /home on your local machine (Ansible server)
2. Find those whose age is equal to or less than 1 week.
3. Find those whose size is equal to or less than 1m.
4. Find the files recursively and set the file type to file.

یه نکته درباره delegad to تو PlayBook اگه بخواهیم یه step روی یه host انجام نشود استفاده می کنیم

فرمش با --limit این هست این میاد کل task ها رو محدود می کنه این فقط یه task رو محدود می کنه



Conditions

شرط گذاشتن روی task ها

```
- set_fact: package_name=httpd
  when: ansible_os_family=="Redhat“

- set_fact: package_name=apache2
  when: ansible_os_family=="Suse"


- name: check if cpu type supported!
  debug: msg="The cpu type is not supported"
  when: ansible_machine != "x86_64"
```




Conditions based on registered value

```
- name: Quering rpm list for apache2 package
  shell: rpm -qa | grep apache2
  register: apache2_rpm

- name: check if apache2-prefork is installed
  debug: msg="apache2-prefork is installed"
  when: "'apache2-prefork' in apache2_rpm.stdout"
```




Conditions

کل اطلاعات پکیج ها رو می فرستد

```
- name: Gather the rpm package facts
  package_facts:
    manager: auto
  tags: cond2

- name: check if nginx is already installed
  debug:
    var: ansible_facts.packages['nginx']
  tags: cond2

- name: if nginx is installed
  debug:
    msg: "nginx is installed"
  when: "'nginx' in ansible_facts.packages"
  tags: cond2
```

Manager
Auto, rpm, apt



Conditions

```
- name: check if a variable defined!
  pause: prompt="The backup path is undefined! Please provide a path for backup"
  register: backup_path2
  when: backup_path is not defined
```

Pause

Pauses playbook execution for a set amount of time, or until a prompt is acknowledged.

prompt

prompt the user for certain input



Exercise 28: conditions

1. Install nginx only on your centos 7 remote hosts
2. Run rpm -qa and check if nginx-mod-mail is installed
3. Check if /home/dir1 exists
   - Use stat module



Exercise 28: conditions (solved)

```
- name: ----- condition -----
  yum:
   name: nginx
   state: present
  when:
   - ansible_distribution == "CentOS"
   - ansible_distribution_major_version == "7"
  tags: [cond1]

- name: ----- CONDITION -----
  shell: "rpm -qa | grep nginx-mod-mail"
  register: rpm_out
  tags: [cond1]

- name: ----- condition -----
  debug:
   msg: 'nginx-mod-mail is installed'
  when: "'nginx-mod-mail' in rpm_out.stdout"
  tags: [cond1]
```

```
- name: ----- condition -----
  stat:
   path: /home/dir1
  register: stat_out
  tags: [cond1]

- name: ----- condition -----
  debug:
   msg: 'directory exists'
  when: stat_out.stat.exists
  tags: [cond1]
```




Conditions based on registered value

```
- name: ----- cat /home/test.txt -----
  shell: cat /home/test.txt
  register: catty
  tags: cond3

- name: ----- if /home/test.txt contains the word Ansible ------
  debug:
      msg: "the test.txt contains the word Ansible :)"
  when: catty.stdout.find('Ansible') != -1
  tags: cond3
```




Conditions based on registered value

```
vars:
  var1: true
  var2: "yes"
-----------------------
- name: - - - if "var1" or "var2" is true - - -
  shell: echo “var1 !"
  when: var1 or var2 | bool

- name: - - - if var1 is false - - -
  shell: echo “var1 is false"
  when: not var1
```

✓ Apply the | bool filter to non boolean variables, such as string variables with content like ‘yes’, ‘on’, ‘1’, or ‘true’.



Exercise 29: confirm file deletion

1. Ask user to delete /home/test.txt
2. If the answer is no you must skip deleting the file
3. If the answer is yes then delete the file on both remote hosts

Hint: use the following condition to delete the file

when: hostvars[play_hosts.0].confirm_delete.user_input | bool



Loops (standard loop)

✓ Sometimes you want to repeat a task multiple times, So you can use loop structure in Ansible.

```
- name: standard loop
  file:
    path: /home/{{ item }}
    state: touch
    owner: root
    group: root
    mode: 0644
  with_items:
    - a1
    - a2
```




Exercise 29: confirm file deletion (solved)

```
- name: - - - - - Confirm file deletion - - - - -
  pause:
    prompt: "Are you sure you want to delete /home/test.txt? (yes/no)"
  register: confirm_delete
  tags: [file_del]

- name: Delete backup file
  file:
    path: /home/test.txt
    state: absent
  when: hostvars[play_hosts.0].confirm_delete.user_input | bool
  tags: [file_del]
```




Loops (standard loop)

✓ Sometimes you want to repeat a task multiple times, So you can use loop structure in Ansible.

```
- name: standard loop
  file:
    path: /home/{{ item }}
    state: touch
    owner: root
    group: root
    mode: 0644
  with_items:
    - a1
    - a2
```



Loops (standard loop)

```
- name: standard loop
  yum:
    name: ['httpd','gcc']
    state: present
```

```
- name: standard loop (deprecation)
  yum: name={{ item }} state=present
  with_items:
    - apache2
    - gcc
```



Loops (standard loop)

```
- name: standard loop
  file:
    path: /home/{{ item }}
    state: touch
    owner: root
    group: root
    mode: 0644
  loop:
   - a1
   - a2
```

```
- name: standard loop
  yum:
    name: {{ item }}
    state: present
  loop:
   - httpd
   - gcc
```




Exercise 30: standard loop

1. Create a playbook to setup the following structure on your ansible server (localhost) (use loop and variables)

```
.
├── inventory
│   └── anisahosts
├── myproject.yml
└── roles
    └── myproject
        ├── defaults
        │   └── main.yml
        ├── files
        ├── handlers
        │   └── main.yml
        ├── meta
        │   └── main.yml
        ├── tasks
        │   └── main.yml
        ├── templates
        └── vars
            └── main.yml
```




Exercise 30: standard loop (solved)

```
- name: ----- create project yml -----
  file:
   path: "{{ansible_root}}/{{project_name}}"
   state: touch
  tags: [create_prj_yml]

- name: ----- create project role -----
  file:
   path: "{{ansible_root}}/roles/{{project_name}}"
   state: directory
  tags: [create_role]

- name: ----- create 7 directories -----
  file:
   path: "{{ansible_root}}/roles/{{project_name}}/{{item}}"
   state: directory
  loop:
   - tasks
   - vars
   - templates
   - handlers
   - files
   - defaults
   - meta
  tags: [seven_dir]
```

```
- name: ----- create main.yml files -----
  file:
   path: "{{ansible_root}}/roles/{{project_name}}/
{{item}}/main.yml"
   state: touch
  loop:
   - tasks
   - vars
   - handlers
   - defaults
   - meta
  tags: [create_main_yml]
```




Exercise 31: create multiple users

1. Define a variable in vars/main.yml named user_lists (user1,user2,user3)
2. Create the three users on remote hosts using loops
3. Change the state to absent to delete the users



Exercise 31: create multiple users (solved)

vars/main.yml:

```
user_lists:
    - "user1"
    - "user2"
    - "user3"
```

tasks/main.yml

```
- name: ----- create multiple users -----
  user:
    name: '{{item}}'
    state: present
  loop: '{{user_lists}}'
  tags: [loop3]
```



Loops (standard loop)

✓ Manage multiple packages with different states

```
- name: - - - - - standard loop - - - - -
  yum:
    name: '{{ item.name }}'
    state: '{{ item.state }}'
  with_items:
    - { name: 'apache2' , state: 'latest' }
    - { name: 'gcc' , state: 'absent' }
```



Loops (standard loop)

✓ Manage multiple packages with different states

```
- name: - - - - - standard loop - - - - -
  yum:
    name: '{{ item.name }}'
    state: '{{ item.state }}'
  with_items:
    - { name: 'apache2' , state: 'latest' }
    - { name: 'gcc' , state: 'absent' }
```



Loops (standard loop)

✓ Create multiple databases via loop

```
- name: --- create multiple databases ---
  mysql_db:
    name: '{{ item }}'
    state: present
  with_items:
    - 'clientdb'
    - 'employeedb'
    - 'providerdb'
  tags: [create_mysql_dbs]
```



Loops (standard loop)

✓ Create multiple mysql user via loop

```
- name: --- Add mysql user ---
  mysql_user:
    name: '{{ item }}'
    password: 123456
    state: present
  with_items:
    - sayahi
    - parsa
  tags: [add_mysql_users]
```




Loops (Nested loop)

✓ Perform multiple operations on the same resource

```
- name: Nested Loop --- give users access to multiple databases
  mysql_user:
    name: '{{ item[0] }}'
    priv: '{{ item[1] }}.*:ALL'
    append_privs: yes
    password: 12345
    login_user: root
  with_nested:
    - ['sayahi' , 'parsa' ]
    - [ 'clientdb','employeedb','providerdb' ]
```


